

TECHNICAL ASSIGNMENT · SOFTWARE ARCHITECT

tv-insight

Clean Architecture on a small scope — and an AI feature with no AI in the use case.

Alex Gomes · alex@agenteresolve.com.br

React 19 · FastAPI · PostgreSQL — `make up` → localhost:7777

THE BRIEF

A deliberately small app — that is the test.

- Four features: **search**, **watch-tracking**, **comments**, **one AI insight**. The features are the easy part.
- The hard questions are at the **seams**: where do vendor failures go, how do you bound a model bill, how do you test without the network?
- So the budget went to the boundaries. Every decision ahead is justified by **what it makes testable**.

Agenda

01 · The one decision

Clean Architecture and the dependency rule you can verify in ten seconds.

02 · Four layers

What each layer owns, and what that ownership buys.

03 · The AI feature

A provider-agnostic port, a domain rule, an honest fallback.

04 · Data, limits & process

Degradation, caching, where it would break, and how I worked with the client.

05 · Ops and tests at four levels · appendix for peer review.

In one slide

- Search TV series on the public **TVMaze API**; browse episodes by season.
- Mark episodes **watched** — the state persists. Comment on a series or an episode.
- An **AI insight** for either, behind a provider-agnostic port with a real fallback strategy.
- Two containers, one command: `make up`.

386

BACKEND TESTS

95

COMPONENT TESTS

11

E2E (CHROMIUM)

321_{MB}

IMAGE, NON-ROOT

The one decision everything follows from

Clean Architecture, four layers, with the **dependency rule** enforced by import direction.

Nothing points outward. The domain imports **only the standard library**.

```
# verify it in ten seconds
grep -rn "^import\|^from" \
  src/tv_insight/domain \
  | grep -v tv_insight
```

```
presentation → application → domain      (nothing points outward)
infrastructure —————
```

That is why the domain tests need no database, no HTTP and no fixtures – and run in milliseconds.



What lives where — and what it buys

LAYER	OWNS	BUYS
domain	invariants, season grouping, watch arithmetic, what an insight may be based on	rules testable with zero infrastructure
application	one class per use case, and the ports	features read as sequences of business steps
infrastructure	TVMaze adapter, LLM adapters, SQL, composition root	vendors are replaceable and absent from the inner layers
presentation	HTTP contract, status codes, cookie identity, the SPA	the framework lives only at the edge

Make invalid states unrepresentable

```
SeriesId(0)           # InvalidValue  
CommentText(" ")      # InvalidValue  
Comment(target=SERIES,  
        episode_id=42) # InvalidValue
```

The `Comment` invariant lives in the entity, so no use case has to remember it.

One translation point maps domain errors → application errors → HTTP: **400 · 404 · 502 · 503**.

Request flow: the series detail screen

```
GET /api/series/169
├─ TVMaze /shows/169          (TTL cache 300s)
├─ TVMaze /shows/169/episodes (TTL cache 300s)
├─ SELECT watched_episodes    ┌
├─ SELECT comments            │─ one transaction (Unit of Work)
└─ SELECT count(comments)     └
↓
domain: WatchProgressService → per-season and overall progress
↓
one JSON response → one round trip → one loading state → one error state
```

Trade-off (D5): the use case knows about four things; the UI gets one call.

SECTION 03

The AI feature

...and why the use case contains no AI.

A port, a domain rule, and a chain

```
GenerateInsight
├─ build InsightSubject
│   (summary, genres, comments)
├─ InsightPromptComposer
│   ← a DOMAIN service
└─ provider.generate(request)
    ← a PORT
```

```
FallbackInsightProvider
├─ HuggingFace (needs a token)
└─ heuristic (always answers)
```

The **rule** (what an insight may be based on) lives in the domain. The **vendor** lives outside, behind a port.

Adding a vendor is **one class plus one registry line**. No use case, port or test double changes.

Prompt text as domain is the debatable part – the rule it encodes is not.

The decision matrix, in order

MODEL PROVIDERS	STORED INSIGHT	ANSWER
answered	any	the fresh answer — persisted as last known good
failed	exists	the stored answer (<code>cached</code> , <code>degraded</code> , with a note)
failed	empty	the heuristic (<code>degraded</code>)

Every **explicit** click tries the providers first. Nothing is requested on page load. Only model answers are persisted.

Failure is a first-class outcome

- `InsightResult` reports **provider**, **degraded**, **cached** and **notes**.
- The UI says “*via heuristic — offline fallback*”. No pretending.
- 429 is reported immediately and the chain moves on — degrading fast beats waiting.
- Catalogue retries transport errors only, never a 500 (that is an answer).
- Upstream errors are normalised in the adapter; no `HttpException` exception reaches a use case.

Caching without a stampede

`AsyncTtlCache.get_or_create` takes a **per-key asyncio lock**: 25 concurrent identical requests trigger **one** upstream call. Asserted in `test_cache.py`.

- Catalogue reads: TTL 300s — TVMaze is public and rate-limited.
- Insights: TTL 900s keyed on the prompt hash. **Regenerate** bypasses with `?refresh=true`.

TABLE	KEY / INDEX
comments	(series_id, target, created_at)
comments	(episode_id, created_at)
watched_episodes	(viewer_id, episode_id) PK
watched_episodes	(viewer_id, series_id)

timestampz . Alembic in the entrypoint . `alembic check` `clean` .
one tx per use case.

Where it would break, and why I built it this way anyway

- **Cache is per process.** The asyncio lock does not cross replicas: N pods = N upstream calls. Redis is the fix.
- **Migrations in the endpoint.** `alembic upgrade head` on every replica can race; production moves it to a release job.
- **Coverage is not quality.** The domain is now **mutation-tested** (226 of 335 mutants killed, ~68%); property testing is still missing.
- **Auth is a guest cookie by design.** `httpOnly`, `SameSite=Lax`; production wants OIDC, and identity is already isolated.

Each of these is a documented trade-off (`docs/TRADEOFFS.md`), not an accident.



The architecture bent; it did not break

CLIENT QUESTION	ANSWER	WHAT CHANGED
Fire insights on page view?	On the button	idle panel + explicit action; cost bounded by intent
Cache until N comments?	"1 is enough"	rule kept; default reversed to 0 = regenerate; store became the <i>failure</i> fallback
One container? through the backend?	"up to you"	single container, same origin; secrets + cache + one error vocabulary stay server-side

6 written questions, each mapped to the file that implements it (docs/CLIENT-FEEDBACK.md). The reversal cost: one environment variable.

One command, two containers

```
docker compose up --build -d # make up
```

```
Dockerfile stage 1 node:22
    npm ci && npm run build
stage 2 python:3.12-slim
    alembic upgrade head
    uvicorn → :7777
non-root · healthcheck · ~321 MB
```

- `depends_on: service_healthy` + `pg_isready` → the app never races the database.
- SPA and API share an origin → no CORS, and the guest cookie behaves in dev and prod.

Testing at four levels — and what it does not prove

LEVEL	WHAT IT PROVES	COUNT
Domain unit	invariants, grouping, progress, prompt rules, insight freshness	97
Use case unit	every use case against hand-written fakes, including the insight store	58
Infrastructure unit	mappers, HTTP adapter, cache, provider chain, heuristic, config	170
Presentation + integration	app factory, session, the real ASGI contract, PostgreSQL repositories	61
System / E2E	the built bundle in Chromium against the real API and database	11

Fakes implement the real ports, so a port change breaks the suite. The contract is generated from **OpenAPI** (Swagger UI at `/api-docs`) and CI fails on drift; the domain is mutation-tested (non-blocking, **~68% killed**). `ruff`/`mypy --strict` are a gate, not a metric.

If I had another week

- 01 Move the cache and the insight store to **Redis** — make the app horizontally honest.
- 02 Migrations as a **release job** + a smoke test against a canary.
- 03 Raise the domain **mutation score** (kill the 109 survivors) and add **property tests**.

Thanks — **Alex Gomes** · alex@agenteresolve.com.br

make up → localhost:7777

Appendix · the five files to read

- 01 `infrastructure/composition.py` — the entire object graph in one place.
- 02 `domain/entities/guide.py` — pure business logic (`seasons()`).
- 03 `application/use_cases/generate_insight.py` — the AI feature with no AI in it.
- 04 `infrastructure/ai/fallback.py` — the fallback composite.
- 05 `tests/integration/test_api.py` — the contract, end to end, in process.